

DAY 15: FULL-STACK INTEGRATION

Naal 15 — Week 3 finale — frontend, backend serthu, oru real app!



CORS



Auth tokens



Integration debugging



Full-stack app

Week 3 · Day 15 of 20 · Rajapalayam · **Week 3 finale**

RECAP DAY 14 — TODAY'S MAP

*Netraikku authenticated API build pannitanga — indru
real UI-oda connect pannுலோம்*

✓ You now have two separate projects:

- A React frontend (Week 2) — beautiful, but not connected to real, protected data
- An authenticated Express + MongoDB API (this week) — powerful, but no user interface

Today, they become one application.

 Today's five blocks:

1 Running two projects at once + CORS

2 Environment variables — on the frontend, this time

3 Working with protected endpoints

4 Common integration pitfalls

5 Bringing it all together + homework

1

BLOCK 1

CONNECTING TWO SEPARATE PROJECTS

Rendu servers, oru neram — pudhu mental model

THE MENTAL SHIFT — TWO SERVERS RUNNING AT ONCE

*Ivvaraikkum oru project mattum run pannitanga.
Indru, rendu*

Terminal 1 — frontend

```
$ npm run dev  
Local: http://localhost:5173
```



Terminal 2 — backend

```
$ npm run dev  
Connected to MongoDB!  
Server on http://localhost:5000
```

Two separate processes, two separate ports. Your React app in the browser makes fetch calls from `localhost:5173` to `localhost:5000` — genuinely two different addresses.

CORS — DAY 7'S ERROR, **FIXED TODAY**

Adhu neenge fetch pannina paartha error — indru, un sondha server-a fix pannu வேண்டும்

●●● Console

```
Access to fetch at 'http://localhost:5000/tasks'
from origin 'http://localhost:5173' has been
blocked by CORS policy
```

Day 7: "not your fault — the API isn't built for browsers." Today: *you* are the API.

```
const cors = require("cors");
app.use(cors());
```

One middleware line — Day 12's pattern

Tells the browser "requests from other origins are welcome here." `npm install cors` first.

2

BLOCK 2

ENVIRONMENT VARIABLES ON THE FRONTEND

*Day 13, 14-la backend .env kathukiteenga —
frontend-ku vera system irukku!*

TWO DIFFERENT SYSTEMS — **DON'T MIX THEM UP**

process.env (backend) vs import.meta.env (frontend, Vite)

Backend (Node) — Day 13/14

```
# .env
MONGODB_URI=...
```

Read with `process.env.MONGODB_URI`. Never sent to the browser — stays server-side, secret.

≠

Frontend (Vite) — today

```
# .env
VITE_API_URL=http://localhost:5000
```

Read with `import.meta.env.VITE_API_URL`. Bundled into your app — visible to anyone, never for real secrets.

Critical rule: a Vite `.env` variable is only exposed to your code if its name starts with `VITE_`. This is a safety feature, not an inconsistency.

POINTING **FETCH** AT YOUR REAL BACKEND

Hardcoded URLs—a badhila — one env variable, everywhere

my-react-app/

```
.env ← VITE_API_URL=http://localhost:5000
```

```
const API_URL = import.meta.env.VITE_API_URL;  
const response = await fetch(`${API_URL}/tasks`);
```

Day 6's template literal, real work

Build every URL from one variable, instead of typing `localhost:5000` in ten different files.

Why this actually matters

When you deploy in Week 4, your backend's real address will change from `localhost` to a live URL — you'll change *one* value, not every fetch call.

3

BLOCK 3

WORKING WITH PROTECTED ENDPOINTS

Day 14-oda token-a, ippo real UI-la use pannu வேளம்

STORING THE JWT — LOCALSTORAGE

Login success aana, token-a browser-la vachukonga

```
async function handleLogin(email, password) {  
  const res = await fetch(`${API_URL}/login`, {  
    method: "POST",  
    headers: { "Content-Type": "application/json" },  
    body: JSON.stringify({ email, password })  
  });  
  const data = await res.json();  
  localStorage.setItem("token", data.token);  
}
```

localStorage — the browser's small filing cabinet

Saves the token even after the tab closes, so users stay logged in on their next visit.

Full circle from Day 14

`data.token` is exactly what `jwt.sign()` sent back yesterday. Same token, now on the other side of the wire.

SENDING THE TOKEN — AUTHORIZATION HEADER

Day 14-oda requireAuth, exact-a idha expect pannuthu

```
async function deleteTask(id) {  
  const token = localStorage.getItem("token");  
  await fetch(`${API_URL}/tasks/${id}`, {  
    method: "DELETE",  
    headers: { Authorization: `Bearer ${token}` }  
  });  
}
```

Exact match: Day 14's middleware read `req.headers.authorization`, expecting `"Bearer <token>"`. This is that string, built on the client.

CONDITIONAL UI — LOGGED IN OR NOT

Day 10-oda ternary pattern — auth state-ku apply pannுலோம்

```
const [token, setToken] = useState(  
  localStorage.getItem("token")  
);  
  
{token  
  ? <Dashboard />  
  : <LoginForm onLogin={setToken} />}
```

useState's initial value — Day 9

Reading directly from localStorage when the component first loads means a refreshed page still remembers you're logged in.

Day 10's ternary, doing real work

Have a token? Show the real app. Don't? Show the login form. One line, the entire app's logged-in/out state.

4

BLOCK 4

COMMON INTEGRATION PITFALLS

Rendᳵ projects serndhu velai seiyoᳵ᳚ podhu, pudhu vidhamana bugs varum

THE INTEGRATION **DEBUGGING CHECKLIST**

Day 7, 9-oda habits — indru rendu projects-ku apply pannu

"Nothing happens" or CORS error

Is the backend actually running? Check terminal 2.
Is cors () middleware installed? Is API_URL correct?

401 Unauthorized

Is a token actually in localStorage? Is the header spelled exactly "Bearer <token>"? Did the token expire?

Browser Network tab — your best friend

See the exact request URL, headers, and response for every fetch — Day 7's "check the real response" habit, applied live.

Symptom-first, still (Day 9)

"Clicking delete does nothing" — before touching code, describe exactly what you clicked, expected, and saw.

5

BLOCK 5

BRINGING IT ALL TOGETHER

20 naal-oda ellaa skills-um — oru real app-la

THE FULL PICTURE — FRONTEND TO DATABASE

15 naal-oda ellaa piece-um, ippo oru diagram-la

1

React UI — user types, clicks (Weeks 1–2)

2

fetch() — with Authorization header (Days 6, 14, 15)

3

Express — CORS → logging → requireAuth middleware (Days 12, 14, 15)

4

Route handler — ownership check, business logic (Day 12, 14)

5

Mongoose — real, permanent MongoDB storage (Day 13)

6

Response → React state → screen updates (Days 9, 10)

LIVE WALKTHROUGH — ONE FEATURE, END TO END

Together-a, oru feature-a mudhal irundhu kadaisi varaikkum build pannuvelam

1

Add `cors()` and `VITE_API_URL` — confirm both servers can talk (Blocks 1-2)

2

Build a real login form; store the returned token (Block 3)

3

Fetch the logged-in user's tasks with the token attached

4

Render them with Day 10's `.map()` + key

5

Click delete — watch a real 403 if it's not your task, or real success if it is

This exact sequence — one feature, wired completely end to end — is your template for tonight's homework.

WRITING YOUR **README** — HOW TO RUN BOTH HALVES

*Day 12-la endpoints document pannitanga — indru
full project document pannுலோபம்*

Section	Contents
Setup	clone, npm install (both folders)
Backend .env	MONGODB_URI, JWT_SECRET (names only, not real values!)
Frontend .env	VITE_API_URL
Run	2 terminals: npm run dev in each folder

Test this yourself: could someone who has never seen your project follow only your README and get it running? That's the real bar.

6

BLOCK 6

AI FOCUS — STEP-BY-STEP INTEGRATION PLANNING

*Rendõ projects-a wire pannõradhu — Al-kitta exact
sequence kekkunga*



STEP-BY-STEP INTEGRATION PLANNING

*Wiring order-a AI-kitta plan pannachonga —
CORS/network errors-ku troubleshoot pannachonga*

Ask AI to lay out the exact sequence of steps to wire frontend to backend. Use it to troubleshoot CORS and network errors as they come up.

✗ Trial and error

Randomly change code in both projects until something works, without understanding why it was broken.

Often "fixes" things by accident, leaving a subtly broken setup that fails again later.

✓ Integration-planning prompt

"I have a React app on port 5173 and an Express API on port 5000 with JWT auth. Lay out the exact sequence of steps to connect them, including CORS and token handling."

Gives a real checklist to follow — and a partner for the CORS/network errors that inevitably show up.

7

BLOCK 7

HOMEWORK

Un first, complete, sondha full-stack app

HOMEWORK — DAY 15

*React app-a, un Week 3 API-oda connect pannunga
— full CRUD, login/signup*

Complete all steps:

① **Connect your React app to your API**

Week 2's project, or a new one — with `cors()` and `VITE_API_URL` wired correctly.

② **Build real signup and login forms**

Storing the token, with conditional UI based on auth state.

③ **Full CRUD, working end to end**

Create, read, update, delete — every request sending the token correctly.

④ **Write a clear README.md**

How to run both halves — tested by actually following your own instructions.

 AI log tonight

Log your integration plan, and any real CORS/network error you hit and how you resolved it.

 Deliverable for Day 16

Working full-stack app (local), pushed to GitHub, plus your README.

 **Week 3 complete after tonight!** Node, Express, MongoDB, authentication — you can now build a complete backend and connect it for real.

Week 3 · Day 15 of 20 · **PuthuTrendInstitute.in** ·
93601-80110



WEEK 3 COMPLETE — YOU BUILT A REAL BACKEND

Ainthu naal — plain Node scripts-la irundhu, full-stack app varaikkᵃᵃᵃ

11

Node.js fundamentals — npm, modules, the file system

12

Express.js — routing, middleware, a real CRUD REST API

13

MongoDB & Mongoose — permanent, validated data persistence

14

Authentication — hashed passwords, JWTs, protected routes

15

Full-stack integration — a complete, connected application

Coming up — Week 4: Testing, debugging, deployment — and your capstone project, live on the internet.

Speaker notes